

C++ 프로그래밍 (STL 컨테이너)

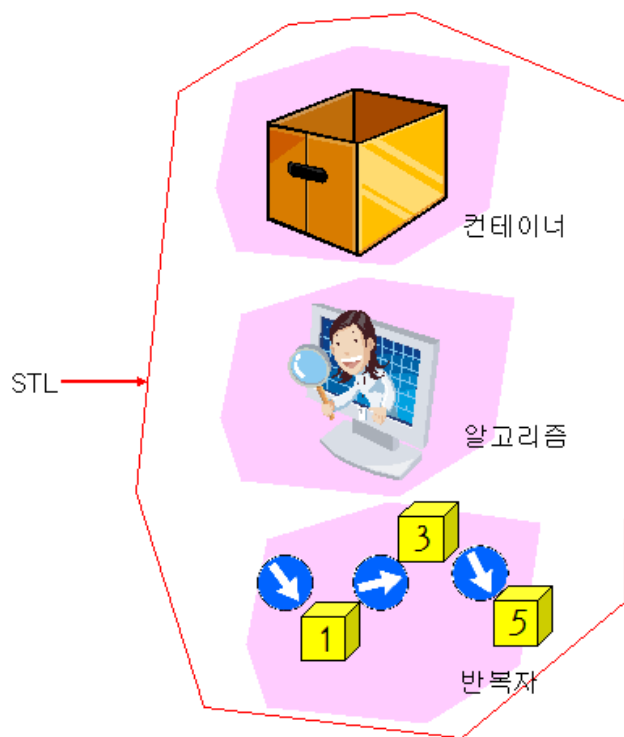
13주차

목차

- STL 소개
- 벡터
- 데크
- 리스트
- 집합
- 맵
- 스택
- 큐

STL 소개

- STL
 - 표준 템플릿 라이브러리(Standard Template Library)의 약자로서 많은 프로그래머들이 공통적으로 사용하는 자료 구조와 알고리즘에 대한 클래스



STL 소개 (계속)

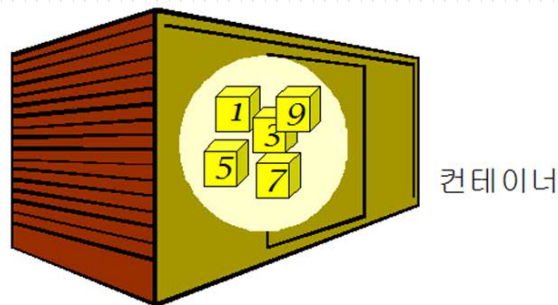
- STL의 3가지 컴포넌트
 - 컨테이너(container)
 - 자료를 저장하는 구조이다.
 - 벡터, 리스트, 맵, 집합, 큐, 스택과 같은 다양한 자료 구조들이 제공된다.
 - 반복자(iterator)
 - 컨테이너 안에 저장된 요소들을 순차적으로 처리하기 위한 컴포넌트
 - 알고리즘(algorithm)
 - 정렬이나 탐색과 같은 다양한 알고리즘을 구현

STL 소개 (계속)

- STL의 장점
 - STL은 전문가가 만들어서 테스트를 거친 검증된 라이브러리
 - STL은 객체 지향 기법과 일반화 프로그램이 기법을 적용하여 만들어졌으므로 어떤 자료형에 대해서도 적용
 - STL을 사용하면 개발 기간을 단축할 수 있고 버그가 없는 프로그램

STL 소개 (계속)

- 컨테이너



분류	컨테이너 클래스	설명	헤더 파일
순차 컨테이너	vector	벡터처럼 입력된 순서대로 저장	<vector>
	list	순서가 있는 리스트	<list>
	deque	양 끝에서 입력과 출력이 가능	<deque>
연관 컨테이너	set	수학에서의 집합 구현	<set>
	multiset	다중 집합(중복을 허용)	<set>
	map	사전과 같은 구조	<map>
	multimap	다중 맵(중복을 허용)	<map>
컨테이너 어댑터	stack	스택(후입선출)	<stack>
	queue	큐(선입선출)	<queue>
	priority_queue	우선순위큐(우선순위가 높은 원소가 먼저 출력)	<queue>

STL 소개 (계속)

- 컨테이너 (계속)
 - 순차 컨테이너 : 자료를 순차적으로 저장
 - 벡터(vector) : 동적 배열처리 동작한다. 뒤에서 자료들이 추가된다.
 - 덱(deque) : 벡터와 유사하지만 앞에서도 자료들이 추가 될수 있다.
 - 리스트(list) : 벡터와 유사하지만 중간에서 자료를 추가하는 연산이 효율적이다.

STL 소개 (계속)

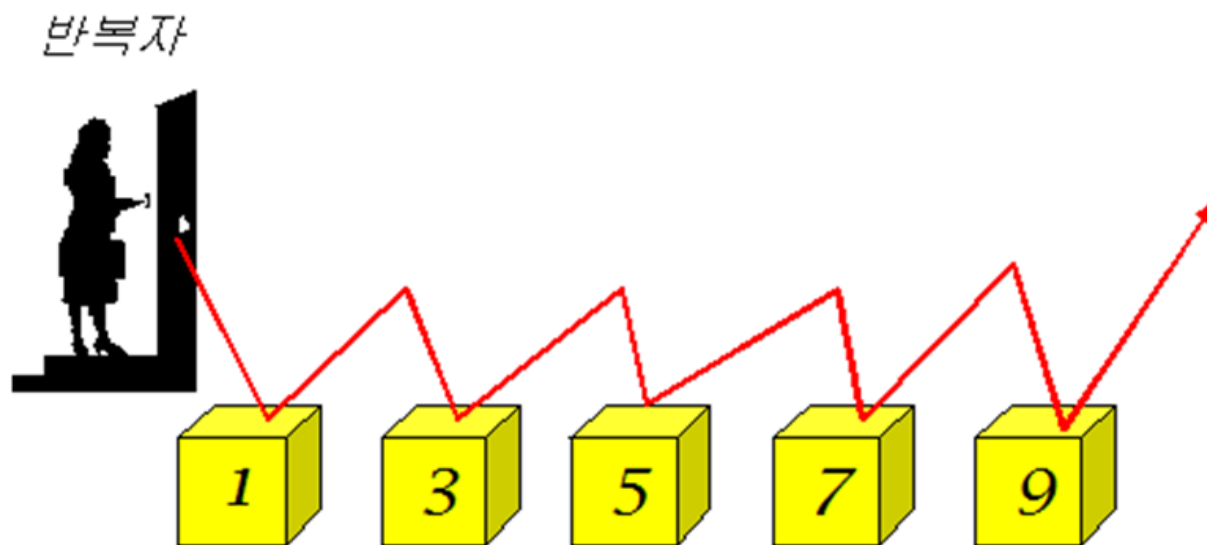
- 컨테이너 (계속)
 - 연관 컨테이너 :
 - 1) 사전과 같은 구조를 사용하여 자료를 저장
 - 2) 원소들을 검색하기 위한 키(key)
 - 3) 자료들은 정렬
 - 집합(set) : 중복이 없는 자료들이 정렬되어서 저장된다.
 - 맵(map) : 키-값(key-value)의 형식으로 저장된다. 키가 제시되면 해당되는 값을 찾을 수 있다.
 - 다중-집합(multiset) : 집합과 유사하지만 자료의 중복이 허용된다.
 - 다중-맵(multimap) : 맵과 유사하지만 키가 중복될 수 있다.

STL 소개 (계속)

- 컨테이너 (계속)
 - 컨테이너 어댑터 : 순차 컨테이너에 제약을 가해서 데이터들이 정해진 방식으로만 입출력
 - 스택(stack) : 먼저 입력된 데이터가 나중에 출력되는 자료 구조
 - 큐(queue) : 데이터가 입력된 순서대로 출력되는 자료 구조
 - 우선 순위큐(priority queue) : 큐의 일종으로 큐의 요소들이 우선 순위를 가지고 있고 우선 순위가 높은 요소가 먼저 출력되는 자료 구조

STL 소개 (계속)

- 반복자(iterator)
 - 현재 처리하고 있는 자료의 위치를 기억하는 객체
 - 포인터와 유사
 - *연산자 사용 가능
 - ++연산자 사용 가능

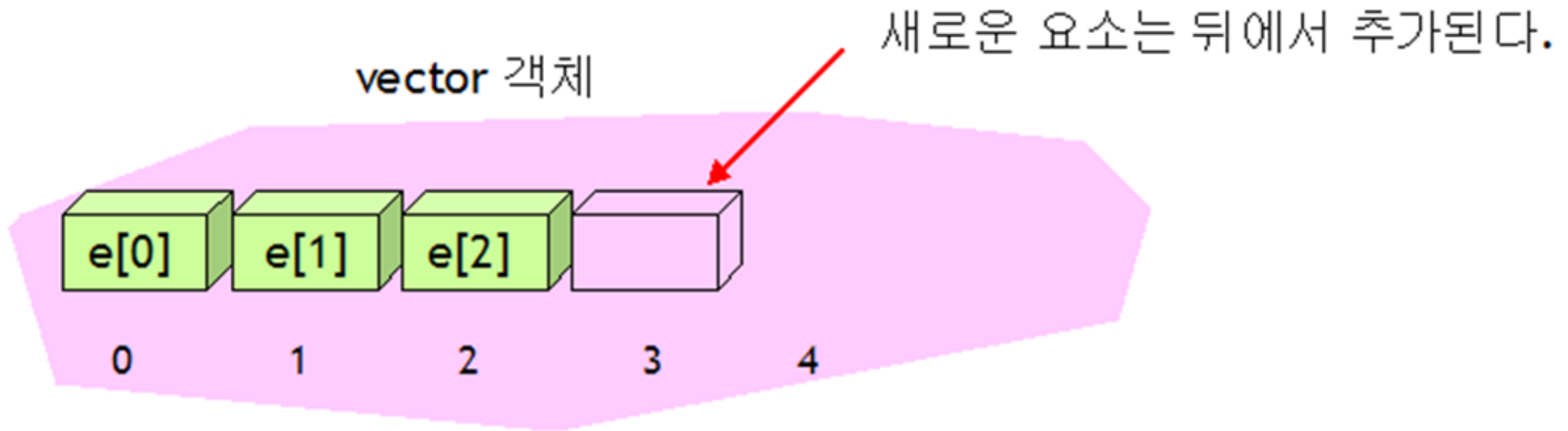


STL 소개 (계속)

- 알고리즘
 - 탐색(find) : 컨테이너 안에서 특정한 자료를 찾는다.
 - 정렬(sort) : 자료들을 크기순으로 정렬한다.
 - 반전(reverse) : 자료들의 순서를 역순으로 한다.
 - 삭제(remove) : 조건이 만족되는 자료를 삭제한다.
 - 변환(transform) : 컨테이너의 요소들을 사용자가 제공하는 변환 함수에 따라서 변환한다.

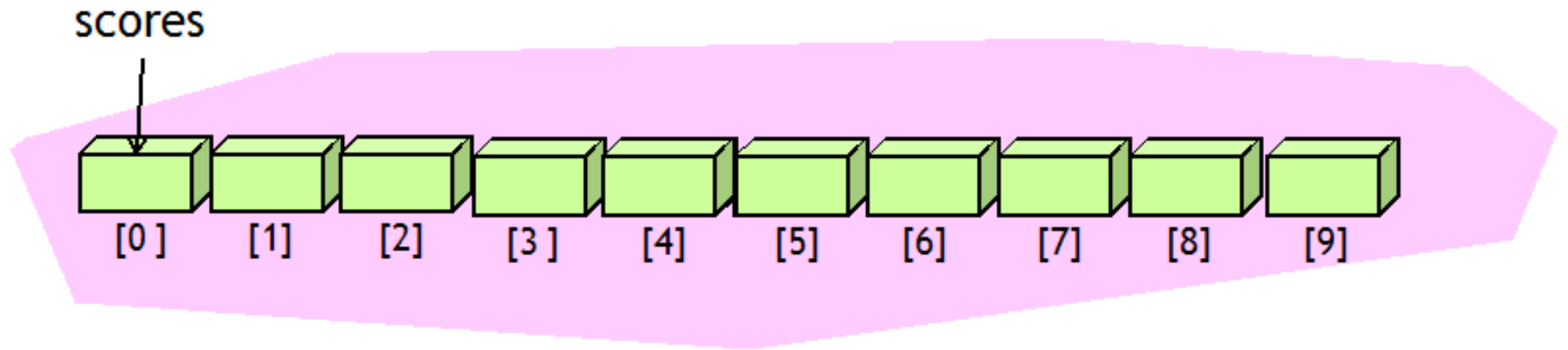
벡터

- 벡터
 - 벡터 == 동적 배열 == 스마트 배열
 - 템플릿으로 설계



벡터 (계속)

- 예제
 - 학생들의 성적을 저장하는 벡터를 생성하여 보자



벡터 (계속)

- 예제 (계속)

```
#include <iostream>
#include <vector>           // 벡터를 사용하려면 이 헤더 파일을 포함하여야 한다
using namespace std;

int main()
{
    vector<double> scores(10);    // 벡터를 생성한다.

    for(int i = 0; i < scores.size(); i++)
    {
        cout << "성적을 입력하시오: ";
        cin >> scores[i];
    }

    double highest = scores[0];
    for(int i = 1; i < scores.size(); i++)
        if( scores[i] > highest )
            highest = scores[i];
    cout << "최고 성적은 " << highest << "입니다.\n";

    return 0;
}
```

실행 결과

성적을 입력하시오: 10

성적을 입력하시오: 20

...

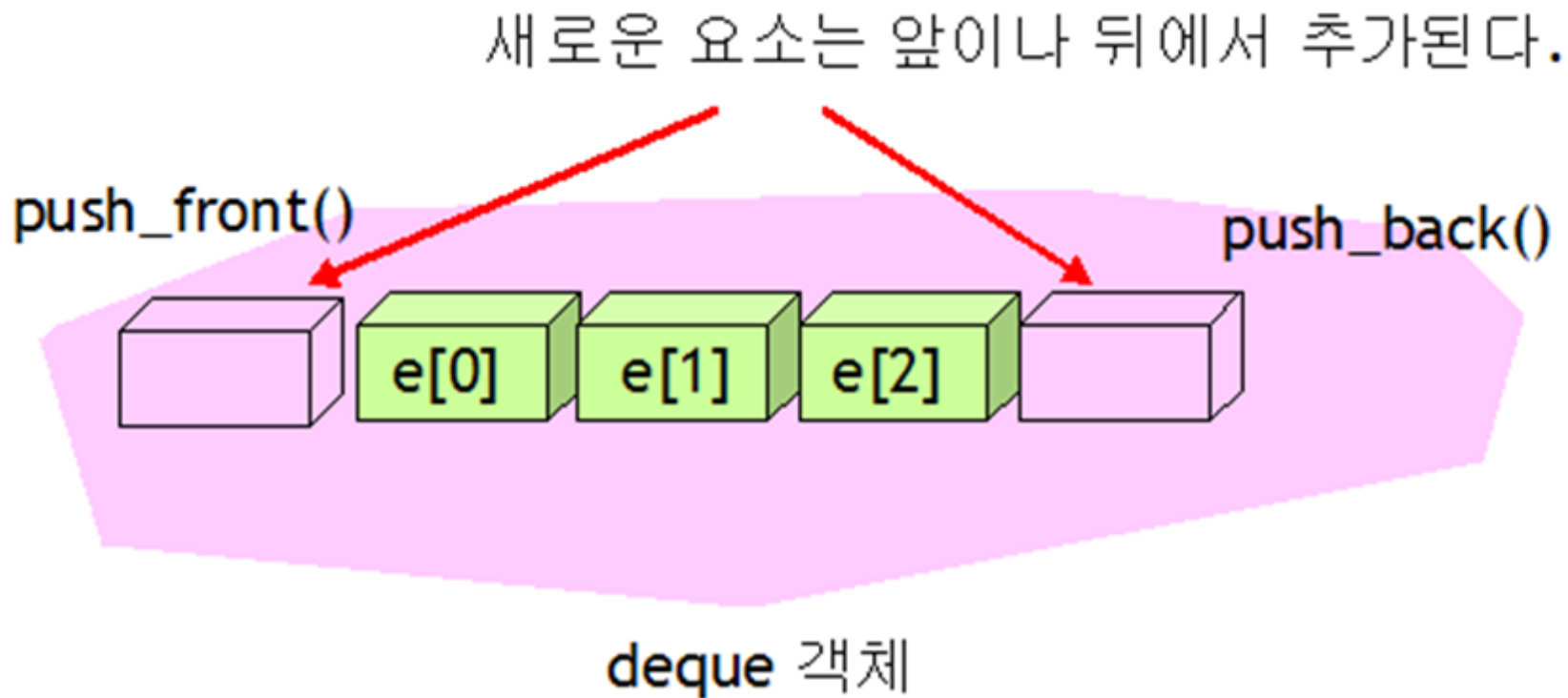
성적을 입력하시오: 90

성적을 입력하시오: 100

최고 성적은 100입니다.

데크

- 데크의 경우, 전단과 후단에서 모두 요소를 추가하고 삭제하는 것을 허용



데크 (계속)

- 예제

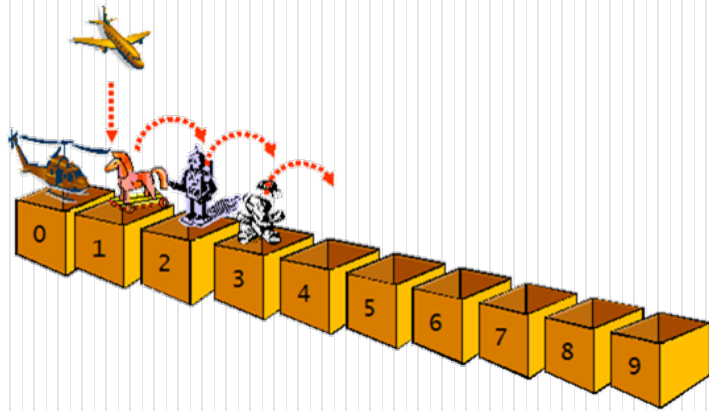
```
int main()
{
    deque<int> dq;
    dq.push_back(99);
    dq.push_back(1);
    dq.push_front(35);
    dq.push_front(67);

    for(int i=0;i<dq.size();i++)
        cout << dq[i] << " ";    // [] 연산자 사용
    cout << endl;
    dq.pop_back();
    dq.pop_front();
    for(int i=0;i<dq.size();i++)
        cout << dq[i] << " ";    // [] 연산자 사용
    cout << endl;

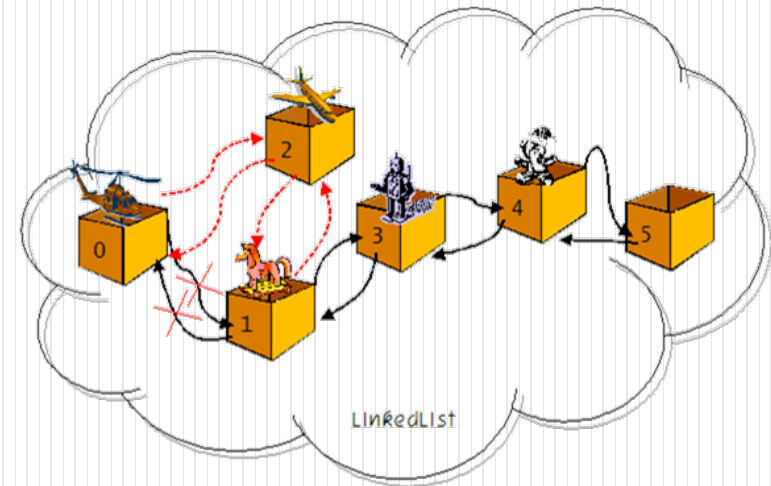
    return 0;
}
```


리스트

- 외부에서 보면 벡터와 동일
- 이중 연결 리스트로 구현



배열의 중간에 삽입하려면 원소들을 이동하여야 한다.



연결 리스트 중간에 삽입하려면 링크만 수정하면 된다.

리스트 (계속)

● 예제

```
void print_list(list<int>& li);
int main()
{
    list<int> my_list;

    my_list.push_back(10);
    my_list.push_back(20);
    my_list.push_back(30);
    my_list.push_back(40);

    my_list.insert(my_list.begin(), 5);
    my_list.insert(my_list.end(), 45);
    print_list(my_list);
    return 0;
}
```

```
void print_list(list<int>& li)
{
    list<int>::iterator it;
    for(it=li.begin(); it!=li.end(); ++it)
        cout << *it << " ";
    cout << endl;
}
```

실행 결과

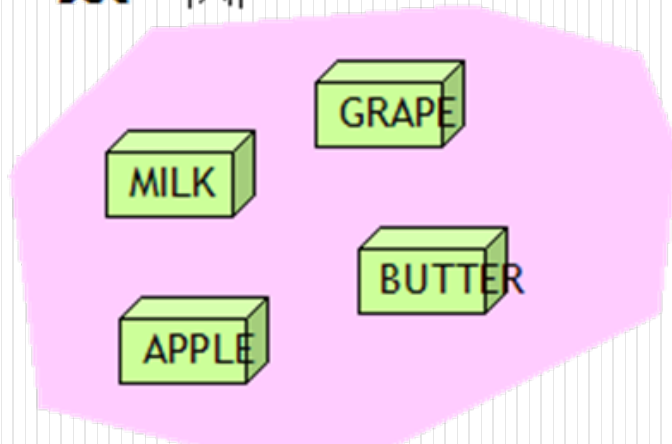
5 10 20 30 40 45

계속하려면 아무 키나 누르십시오 . . .

집합

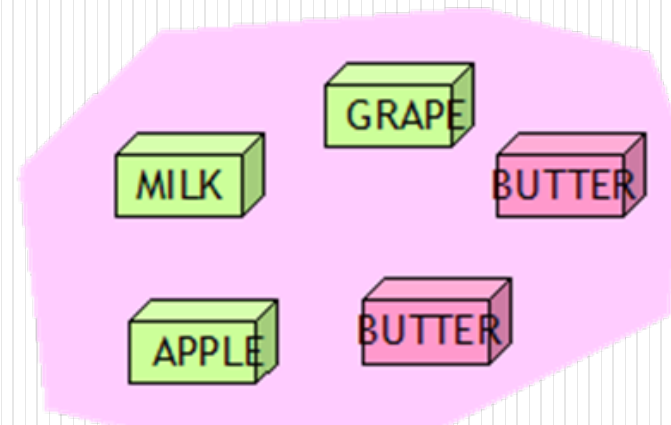
- 수학적인 집합과 비슷
- 집합(set)은 동일한 키를 중복해서 가질 수 없다.
예) $A = \{1, 2, 3, 4, 5\}$ 는 집합이지만 $B = \{1, 1, 2, 2, 3\}$ 은 집합이 아니다.

set 객체



집합은 순서가 없고 중
복을 허용하지 않음

multiset 객체



집합은 순서가 없고 중
복을 허용하지 않음

집합 (계속)

- 예제

```
template <typename T>
void print_list(const T& container);
```

```
int main()
{
    set<int> my_set;
    multiset<int> my_multiset;

    my_set.insert(1);
    my_set.insert(2);
    my_set.insert(3);

    my_multiset.insert(my_set.begin(), my_set.end());
    my_multiset.insert(3);
    my_multiset.insert(4);

    print_list(my_set);
    print_list(my_multiset);
    return 0;
}
```

```
template <typename T>
void print_list(const T& container)
{
    T::const_iterator it;
    for(it=container.begin(); it!=container.end(); ++it)
        cout << *it << " ";
    cout << endl;
}
```

실행 결과

1 2 3

1 2 3 3 4

계속하려면 아무 키나 누르십시오 . . .

맵

- Map은 사전과 같은 자료 구조
- 키(key)가 제시되면 Map은 값(value)을 반환한다.

키(key)



값(value)

Map

맵 (계속)

- 예제

```
#include <iostream>
#include <string>
#include <map>
using namespace std;
```

```
int main()
{
    map<string, string> dic;
    dic["boy"]="소년";
    dic["school"]="학교";
    dic["office"]="직장";
    dic["house"]="집";
    dic["morning"]="아침";
    dic["evening"]="저녁";
```

```
    cout << "house의 의미는 " << dic["house"] << endl;      // 등록이 된 단어
    cout << "morning의 의미는 " << dic["morning"] << endl;   // 등록이 된 단어
    cout << "unknown의 의미는 " << dic["unknown"] << endl;  // 등록이 안된 단어
    return 0;
}
```

실행 결과

house의 의미는 집

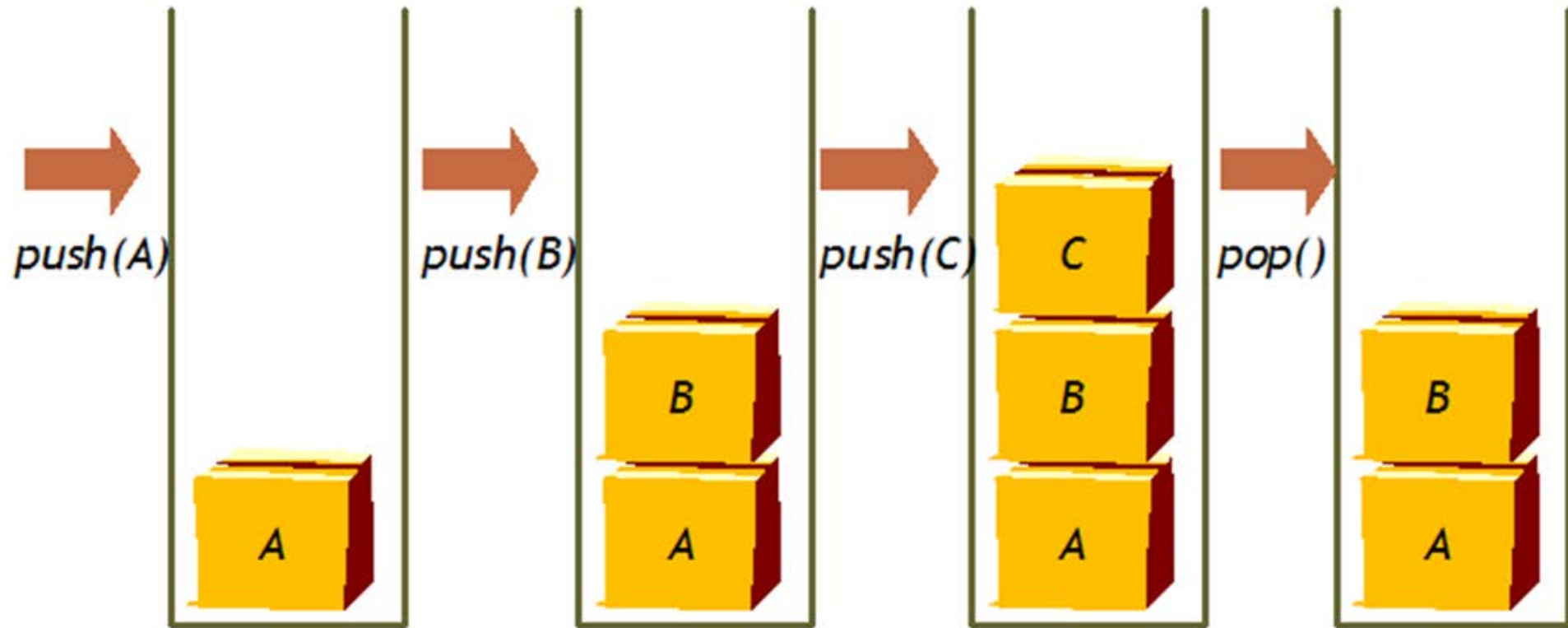
morning의 의미는 아침

unknown의 의미는

계속하려면 아무 키나 누르십시오 . . .

스택

- 스택은 늦게 들어온 데이터들이 먼저 나가는 자료 구조



스택 (계속)

- 예제

// 벡터를 변형하여서 스택을 생성한다.

```
#include <iostream>
```

```
#include <stack>
```

```
#include <string>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    stack<string> st;
```

```
    string sayings[3] =
```

```
    {"The grass is greener on the other side of the fence",
```

```
    "Even the greatest make mistakes",
```

```
    "To see is to believe" };
```

```
    for (int i = 0; i < 3; ++i)
```

```
        st.push(sayings[i]);
```

```
    while (!st.empty()) {
```

```
        cout << st.top() << endl;
```

```
        st.pop();
```

```
    }
```

```
    return 0;
```

```
}
```

실행 결과

To see is to believe

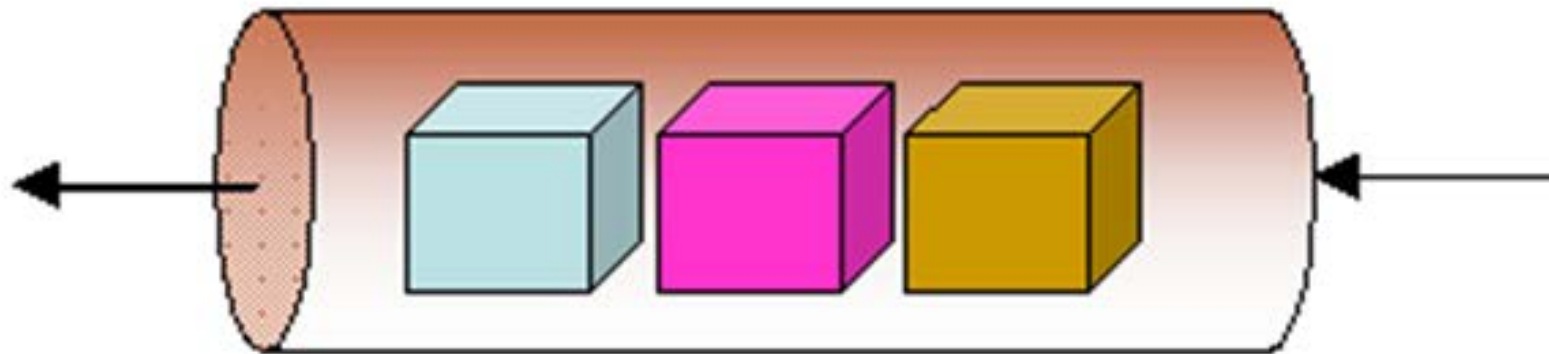
Even the greatest make mistakes

The grass is greener on the other side of the fence

계속하려면 아무 키나 누르십시오 . . .

큐

- 큐(queue)는 먼저 들어온 데이터들이 먼저 나가는 자료구조



전단(head)

후단(tail)

큐 (계속)

- 예제

```
#include <iostream>
#include <queue>
#include <string>
using namespace std;

int main()
{
    queue<int> qu;
    qu.push(100);
    qu.push(200);
    qu.push(300);
    while (!qu.empty()) {
        cout << qu.front() << endl;
        qu.pop();
    }
    return 0;
}
```

실행 결과

100

200

300

계속하려면 아무 키나 누르십시오 . . .